

System Architecture & Feature Overview: Alaya Master Coach

A Full-Stack AI-Driven and Human-Guided Accountability Coaching SaaS Platform

1. Executive Summary

Alaya Master Coach is a modern Software-as-a-Service (SaaS) platform built to foster positive habits and high-impact goal execution. By combining the immediacy of Generative AI (using Groq LLaMA models) with the relational support of a human coach, Alaya provides a hybrid accountability loop.

This document outlines the architecture, database schema, and product features of the platform.

2. System Architecture Design

The platform uses a decoupled, full-stack architecture optimized for low latency, secure authentication, and real-time operations.

```
graph TD
  %% Frontend Layer

  subgraph Frontend [Client & Coach Portal - React 19]
    UI[Tailwind CSS & Radix UI]
    Router[TanStack Router]
```

Query[TanStack React Query]

State[Zustand Store]

WS_Client[SockJS + STOMP Client]

end

%% Gateway/Proxy Layer

subgraph Host [Deployment & Routing]

Cloudflare[Cloudflare / Vite Server]

end

%% Backend Layer

subgraph Backend [Backend API Service - Spring Boot 3]

Sec[Spring Security & JWT]

Controller[REST Controllers]

WS_Server[WebSocket STOMP Broker]

WebClient[Spring WebFlux WebClient]

Service[Business Service Layer]

JPA[Spring Data JPA & Hibernate]

end

%% Database Layer

```
subgraph Storage [Persistent Storage]
```

```
DB[(PostgreSQL Database)]
```

```
end
```

```
%% External Services Layer
```

```
subgraph External [External Services]
```

```
Groq[Groq AI LLaMA API]
```

```
Mail[Java Mail Sender - OTP]
```

```
end
```

```
%% Relationships
```

```
UI --> Router
```

```
Router --> Query
```

```
Query --> Host
```

```
Host --> Sec
```

```
Sec --> Controller
```

```
WS_Client <-->|Real-Time Chat| WS_Server
```

```
Controller --> Service
```

```
WS_Server --> Service
```

```
Service --> WebClient
```

```
Service --> JPA
```

```
JPA --> DB
```

WebClient <-->|Async Non-Blocking| Groq

Service --> Mail

Component Details

- **Frontend Portal (React 19 & TypeScript):** Built with type-safe routing via **TanStack Router**, high-performance server-state synchronization with **TanStack React Query**, and modern responsive design utilizing **Tailwind CSS v4** and **Radix UI** primitives.
- **Backend API (Spring Boot 3 & Java 17):** Handles core business logic, user onboarding, role-based authorization (Client vs. Coach), email dispatch, and database persistence.
- **Asynchronous Integration (Spring WebFlux):** Implements Spring's reactive `WebClient` to call the external Groq API asynchronously. This guarantees that long-running AI feedback generation does not block main application threads.
- **Real-time Engine (SockJS & STOMP):** Utilizes WebSockets to power the chat experience between coaches and clients, enabling direct feedback loops with latency below 100ms.

3. Database Schema

The relational database runs on **PostgreSQL** to manage relational tables, user roles, daily logs, and conversation histories:

| Table / Entity | Primary Fields | Relationships | Purpose |

| **User** | id, email, password, role (CLIENT/COACH), height, weight, calorieGoal | One-to-Many with Checkins, Goals, FoodEntries | Holds client/coach profile and settings |

| **Goal** | id, client_id, description, targetDate, completed | Many-to-One with User
| Stores short/long term accountability targets |

| **Checkin** | id, client_id, date, status, aiFeedback | Many-to-One with User | Logs
daily check-ins; stores AI feedback from Groq |

| **FoodEntry** | id, client_id, foodName, calories, imageUrl, feedback | Many-to-
One with User | Manages calorie/diet logs mapped via AI Vision |

| **ChatMessage** | id, sender_id, recipient_id, content, timestamp | Many-to-One
with User | Saves persistent messaging histories between coach and client |

| **OtpToken** | id, email, otpCode, expiryDate | N/A | Manages secure signup
verification and password resets |

4. Key AI Capabilities

A. Intelligent Accountability Coaching

Every time a client submits a check-in, the system constructs a system prompt outlining behavioral science principles, gathers context on the client's current goals/progress, and sends the package to the **Groq LLaMA** model. The AI returns tailored, empathetic, and direct behavioral suggestions stored on the check-in object.

B. Computer Vision Nutrition Logs

Clients upload meal photographs to track diet goals. The backend:

1. Encodes the image to Base64.
2. Formulates a JSON schema prompt specifying output requirements.
3. Dispatches it to the **Groq Vision LLM**.

4. Receives back parsed JSON fields containing: **Food Name**, **Estimated Calories**, **Nutritional Advice**, and **Dietary Health Classification** (HEALTHY/UNHEALTHY).
-

5. Deployment & Production Setup

- **Frontend:** Optimized for serverless edge platforms (Vercel/Cloudflare Pages).
- **Backend:** Containerized and hosted on cloud virtual instances (Docker, AWS, Railway) communicating with a managed Postgres instance.
- **Filesystem:** Integrates local image storage (standard /uploads) with options for secure S3 buckets.